



UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO

INFORME FINAL — PROYECTO DE SEMESTRE

**APP DE EMPAREJAMIENTO AUTOMATICO DE EQUIPOS
MULTIDISCIPLINARIOS PARA CONVOCATORIAS DE LA
INCUBADORA PAQARINA WASI**

Para optar al título profesional de:
INGENIERO INFORMÁTICO Y DE SISTEMAS

Presentado por:
BR. JOHAN MIHAIL CONDE SALLO

Perú, Agosto del 2026

ESCUELA PROFESIONAL DE INGENIERÍA INFORMÁTICA Y DE SISTEMAS
FACULTAD DE INGENIERÍA ELÉCTRICA, ELECTRÓNICA, INFORMÁTICA Y MECÁNICA

Informe Final del Proyecto

10 de agosto de 2026

Índice

Introducción	4
1. Descripción del Proyecto	5
1.1. Nombre de la aplicación	5
1.2. Descripción del problema	5
1.3. Público objetivo	5
1.4. Objetivo general	6
1.5. Objetivos específicos	6
1.6. Justificación del proyecto	6
1.7. Alcance del informe	7
2. Análisis de Requerimientos	8
2.1. Requerimientos Funcionales (RF)	8
2.2. Requerimientos No Funcionales (RNF)	9
3. Arquitectura de la Solución	10
3.1. Panorama general	10
3.2. Decisión de arquitectura: simplicidad deliberada	10
3.3. Estructura del proyecto	11
3.4. Dependencias principales	12
4. Modelo de Datos	13
4.1. Colecciones en Cloud Firestore	13
4.2. Estrategia de desnormalización	13
4.3. Estrategia de consultas	14
4.4. Modelo lógico relacional	14
5. Motor de Emparejamiento Automático	15
5.1. Modelo conceptual	15
5.2. Función de compatibilidad implementada	15
5.3. Coincidencia difusa de habilidades	16
5.4. Algoritmo de publicación y notificación	16
5.5. Ciclo de vida del ingreso a un equipo	17

6. Seguridad y Control de Acceso	18
6.1. Autenticación	18
6.2. Identidad institucional	18
6.3. Roles de administración	18
6.4. Reglas de seguridad del servidor	18
6.5. Compromiso documentado sobre la escritura de equipos	18
6.6. Gestión de multimedia	19
7. Diseño de Interfaz y Flujo de Navegación	20
7.1. Sistema de diseño	20
7.2. Flujo de navegación	20
7.3. El feed de convocatorias	21
7.4. Pantallas de formación de equipos	21
7.5. Panel de administración	21
7.6. Capturas de pantalla de la aplicación	22
8. Pruebas y Verificación	23
8.1. Estrategia	23
8.2. Casos de prueba automatizados	23
8.3. Resultados de la verificación	24
8.4. Métricas del producto	24
8.5. Verificación manual	24
9. Trazabilidad: Planificado frente a Alcanzado	26
9.1. Desviaciones respecto del plan original y su justificación	28
10.Limitaciones Conocidas	30
11.Trabajo Futuro	31
12.Conclusiones	32
13.Referencias	33

Introducción

Yuyanna Hub es una aplicación multiplataforma (Android y web) desarrollada en Flutter que centraliza las convocatorias de la incubadora universitaria **Paqarina Wasi** de la UNSAAC y automatiza la conformación de equipos multidisciplinarios para postular a ellas. El sistema resuelve un problema de *asimetría de información*: los estudiantes conocen las convocatorias pero no tienen forma ágil de encontrar a los compañeros con las competencias que su proyecto necesita.

La solución implementada consta de tres piezas: (i) un **feed vertical de convocatorias** al estilo de las redes sociales de video corto, con soporte de imagen y video, publicado por administradores de Paqarina Wasi; (ii) un **motor de emparejamiento** que, al publicarse un grupo, recorre el directorio real de usuarios registrados, puntúa a cada candidato contra las habilidades declaradas como faltantes por el equipo y notifica automáticamente a los diez mejores perfiles mediante invitaciones; y (iii) un **ciclo completo de formación de equipos** (borrador → publicación → invitaciones/solicitudes → cierre automático al llenarse el cupo).

El backend se apoya en **Firestore** (Authentication y Cloud Firestore) sobre el plan gratuito *Spark*, complementado con un **Cloudflare Worker** sobre almacenamiento R2 para alojar el material multimedia de los eventos. El producto final cuenta con 36 archivos fuente Dart (aproximadamente 9 770 líneas), 12 pruebas automatizadas que pasan en verde y un análisis estático (**flutter analyze**) sin observaciones.

1 Descripción del Proyecto

1.1 Nombre de la aplicación

Yuyanna Hub

1.2 Descripción del problema

La incubadora de empresas de la Universidad Nacional de San Antonio Abad del Cusco (UNSAAC), **Paqarina Wasi**, gestiona anualmente diversos programas de preincubación, incubación y aceleración de emprendimientos de base tecnológica, así como ferias y fondos concursables orientados a la innovación estudiantil. Estos programas exigen, casi en su totalidad, la conformación de equipos multidisciplinarios sólidos como requisito de postulación.

Sin embargo, la formación de dichos equipos ocurre de manera completamente informal y aleatoria. Las convocatorias de Paqarina Wasi se difunden de forma pasiva y masiva mediante correos institucionales (que el alumnado ignora rutinariamente por saturación de *spam* administrativo) y publicaciones dispersas en redes sociales como Facebook. Como resultado, el estudiante interesado conoce la convocatoria pero no dispone de ningún mecanismo ágil dentro del campus para encontrar a los socios estratégicos que su proyecto necesita.

Esta ausencia de un canal estructurado de emparejamiento desencadena dos problemas principales:

1. **Suboptimización de los fondos de Paqarina Wasi:** las convocatorias de incubación quedan muchas veces desiertas, con baja participación, o reciben propuestas de baja rigurosidad porque los equipos se arman a último momento con compañeros del mismo círculo social y la misma escuela profesional, sin la diversidad de competencias que un emprendimiento real requiere.
2. **Aislamiento de capacidades académicas:** el estudiante con una idea viable no logra postular porque no encuentra los perfiles complementarios que le faltan. Un estudiante de Ingeniería de Sistemas posee la capacidad de arquitectura técnica pero carece de un cofundador de Economía o Contabilidad para estructurar el modelo financiero; un estudiante de Turismo identifica un nicho de mercado real pero no dispone del enlace técnico para automatizar su infraestructura lógica. Como consecuencia, los alumnos recurren a grupos masivos de WhatsApp, limitando el potencial de las *startups* incubadas.

1.3 Público objetivo

- **Usuarios primarios:** estudiantes de pregrado matriculados activamente en las 41 escuelas profesionales de la UNSAAC, prioritariamente de ciclos intermedios y avanzados, interesados en postular a los programas de incubación y ferias de Paqarina Wasi.
- **Usuarios secundarios:** egresados recientes interesados en incubación de base tecnológica, docentes asesores en calidad de mentores técnicos, y los gestores administrativos de Paqarina Wasi encargados de publicar y dar seguimiento a las con-

vocatorias. Estos últimos se modelaron en el sistema como *roles de administración* diferenciados (véase la sección 6).

1.4 Objetivo general

Desarrollar una aplicación móvil multiplataforma llamada **Yuyanna Hub** que centralice las convocatorias de la incubadora **Paqarina Wasi** y conforme equipos multidisciplinarios eficientes mediante un **motor de emparejamiento automático** (*matchmaking*) que cruza el perfil de habilidades de cada estudiante con el déficit de competencias declarado por cada grupo en formación.

1.5 Objetivos específicos

- OE-1.** Diseñar una interfaz móvil minimalista e intuitiva que permita a los estudiantes configurar perfiles que destaquen sus destrezas duras (*hard skills*), su escuela profesional y su semestre.
- OE-2.** Implementar un mecanismo de autenticación y de identidad institucional que distinga a los miembros de la comunidad UNSAAC (dominio `@unsaac.edu.pe`).
- OE-3.** Modelar la representación de las convocatorias de Paqarina Wasi y de los equipos en formación, donde cada equipo declara explícitamente las competencias que le faltan.
- OE-4.** Desarrollar el **motor de emparejamiento automático** que calcule un puntaje de compatibilidad entre un estudiante y las habilidades faltantes de cada equipo, y que notifique proactivamente a los perfiles que mejor cubren ese déficit.
- OE-5.** Proveer un canal de ingreso bidireccional (invitación del sistema y solicitud del estudiante) con control del líder del equipo y cierre automático del grupo al completarse el cupo.
- OE-6.** Dotar a los gestores de Paqarina Wasi de un panel de administración para publicar, editar y dar seguimiento a los eventos, usuarios y grupos.

1.6 Justificación del proyecto

El proyecto se justifica tecnológica y socialmente ya que dinamiza el ecosistema de innovación de la incubadora Paqarina Wasi desde su núcleo estudiantil más grande. Al automatizar el emparejamiento de talento dentro del campus, se maximiza el rendimiento de los fondos públicos asignados a la incubación al garantizar que las propuestas presentadas provengan de equipos completos y balanceados. Desde la ingeniería de software, el motor de *matchmaking* de **Yuyanna Hub** resuelve un problema de asimetría de información y de asignación de recursos humanos, modelándolo como un problema de emparejamiento por afinidad (*matching*) que erradica los vicios de los canales informales.

1.7 Alcance del informe

Este documento reporta el **estado final** del proyecto de semestre. Describe la solución tal como fue construida y desplegada, no únicamente lo planificado: cada requerimiento del plan original se contrasta explícitamente con lo alcanzado (sección 9), y las desviaciones respecto de la propuesta inicial se documentan con su justificación técnica en la sección 10.

2 Análisis de Requerimientos

2.1 Requerimientos Funcionales (RF)

Tabla 1: Matriz de Requerimientos Funcionales de Yuyanna Hub

Código	Nombre	Descripción detallada
RF-01	Autenticación de cuentas	El sistema debe permitir registro e inicio de sesión con correo y contraseña, verificación de correo y restablecimiento de contraseña. Las cuentas del dominio <code>@unsaac.edu.pe</code> se distinguen con la etiqueta <i>Estudiante UNSAAC</i> .
RF-02	Gestión de perfil de habilidades	El usuario debe poder editar su nombre, tipo de perfil (estudiante u externo), escuela profesional, semestre u ocupación, y su conjunto de habilidades etiquetadas. Este perfil alimenta el motor de emparejamiento.
RF-03	Mapeo de convocatorias	El módulo administrador debe permitir registrar, editar y eliminar las convocatorias de la incubadora, categorizándolas por tipo, con fecha de cierre, premio, requisitos e imagen o video promocional.
RF-04	Publicación de equipos y déficit de competencias	El líder debe poder registrar su equipo ligado a una convocatoria, definir su capacidad máxima e indicar explícitamente las habilidades buscadas que le faltan al equipo.
RF-05	Motor de emparejamiento automático	Al publicarse un grupo, el sistema debe calcular un puntaje de compatibilidad entre cada usuario del directorio y las habilidades faltantes del equipo, y notificar a los perfiles de mayor puntaje mediante invitaciones que muestran la compatibilidad detectada.
RF-06	Postulación y aprobación	Un invitado puede aceptar y unirse directamente (la invitación lo pre-aprueba) o rechazar. Un usuario no invitado puede enviar una solicitud de ingreso que el líder acepta o rechaza. Al completarse el cupo, el grupo se cierra automáticamente.
RF-07	Bandeja de notificaciones internas	El usuario debe contar con una bandeja de invitaciones pendientes, con un contador visible en la barra superior del feed.

Código	Nombre	Descripción detallada
RF-08	Panel de administración	Los administradores deben poder gestionar desde un panel los eventos, el directorio de usuarios, los grupos creados y los propios administradores (alta, baja y cambio de rol).
RF-09	Gestión de multimedia	El panel debe permitir subir imágenes y videos de los eventos a un almacenamiento externo y guardar únicamente su URL en la base de datos.

2.2 Requerimientos No Funcionales (RNF)

Tabla 2: Matriz de Requerimientos No Funcionales de Yuyanna Hub

Código	Categoría	Especificación técnica
RNF-01	Multiplataforma	Una sola base de código debe compilar para Android y para navegador web, con la misma interfaz adaptable.
RNF-02	Costo de operación	La solución debe operar íntegramente sobre planes gratuitos (Firebase <i>Spark</i> y Cloudflare <i>free tier</i>), sin costo recurrente para la incubadora durante el piloto.
RNF-03	Eficiencia de lectura	La consulta de un grupo debe traer a todos sus integrantes en una sola lectura de base de datos (sin consultas N+1), y las consultas deben evitar índices compuestos en Firestore.
RNF-04	Seguridad y control de acceso	Las credenciales las gestiona Firebase Authentication (la aplicación nunca almacena contraseñas). El acceso a los datos debe estar reforzado del lado del servidor mediante <i>Security Rules</i> , no solo en el cliente.
RNF-05	Persistencia de sesión	La sesión debe sobrevivir al cierre de la aplicación mediante una caché local del perfil, evitando una lectura remota en cada arranque.
RNF-06	Idioma y consistencia	Todo el dominio, la interfaz y los identificadores del código deben estar en español, acordes al contexto institucional.
RNF-07	Calidad de código	El proyecto debe mantenerse sin observaciones bajo el analizador estático (<i>flutter analyze</i>) y con la lógica de dominio cubierta por pruebas automatizadas.

3 Arquitectura de la Solución

3.1 Panorama general

Yuyanna Hub sigue una arquitectura de **cliente enriquecido sobre backend gestionado** (*serverless* administrado). No existe un servidor propio: la aplicación Flutter conversa directamente con Firebase, y la lógica de dominio —incluido el motor de emparejamiento— se ejecuta en el cliente. El único componente de cómputo propio es un *Cloudflare Worker* dedicado exclusivamente a recibir y servir archivos multimedia.

Tabla 3: Componentes del sistema

Componente	Tecnología	Responsabilidad
Aplicación cliente	Flutter / Dart (SDK ^3.11.4)	Interfaz, navegación, lógica de dominio y emparejamiento.
Autenticación	Firebase Authentication	Registro, inicio de sesión, verificación de correo y restablecimiento de contraseña.
Base de datos	Cloud Firestore (plan <i>Spark</i>)	Persistencia de perfiles, convocatorias, grupos, invitaciones, solicitudes y roles.
Autorización	Firestore Security Rules	Refuerzo de permisos del lado del servidor.
Almacenamiento multimedia	Cloudflare Worker + R2	Recepción, almacenamiento y entrega de imágenes y videos con soporte de <i>Range</i> .
Caché de sesión	<code>shared_preferences</code>	Perfil serializado y bandera de sesión en el dispositivo.

3.2 Decisión de arquitectura: simplicidad deliberada

El proyecto renuncia conscientemente a tres piezas habituales del ecosistema Flutter:

- **Sin paquete de gestión de estado** (Provider, Riverpod, BLoC): cada pantalla es un `StatefulWidget` que administra su propio estado con `setState`. El grafo de estado compartido del piloto es reducido y no justificaba el costo conceptual de una capa adicional.
- **Sin paquete de enrutamiento**: la navegación se resuelve con `Navigator.push` y `MaterialPageRoute`, sin rutas nombradas.
- **Sin generación de código** (`json_serializable`, `freezed`): los modelos son clases inmutables con `fromJson`, `toJson` y `copyWith` escritos a mano, lo que mantiene el ciclo de compilación corto y el código legible sin herramientas intermedias.

A cambio, la arquitectura sí impone una **separación estricta por capas**: los *widgets* nunca consultan Firestore directamente; toda escritura y lectura pasa por un servicio de

lib/services/, y todo el estilo pasa por lib/theme/. Los servicios reciben sus dependencias por constructor ({FirebaseFirestore? firestore, FirebaseAuth? auth}), lo que permite inyectar dobles de prueba (fake_cloud_firestore, firebase_auth_mock) sin tocar servicios reales.

3.3 Estructura del proyecto

```
yuyanna_hub/
|-- lib/
|   |-- main.dart                # Inicializa Firebase y monta la app
|   |-- firebase_options.dart    # Configuración generada por FlutterFire
|   |-- database/
|   |   `-- sq_lite_helper.dart  # LocalSessionManager (shared_preferences)
|   |-- models/                  # Entidades inmutables del dominio
|   |   |-- usuario.dart         # Perfil + habilidades + esUnsaac
|   |   |-- convocatoria.dart    # Evento de Paqarina Wasi
|   |   |-- grupo.dart          # Equipo, cupo y habilidades buscadas
|   |   |-- miembro_resumen.dart # Integrante desnormalizado
|   |   |-- invitacion_grupo.dart # Invitación generada por el matching
|   |   `-- solicitud_ingreso.dart # Postulación de un usuario al grupo
|   |-- services/                # Acceso a datos y lógica de dominio
|   |   |-- auth_service.dart    # Firebase Auth + perfil en Firestore
|   |   |-- convocatoria_service.dart # CRUD de eventos
|   |   |-- grupo_service.dart   # Formación de equipos + matchmaking
|   |   |-- admin_service.dart   # Roles de administración
|   |   |-- habilidades_store.dart # Catálogo de habilidades
|   |   `-- media_upload_service.dart # Subida al Worker de Cloudflare
|   |-- screens/                 # Vistas (StatefulWidget + setState)
|   |   |-- splash_screen.dart   # Carga y decisión de sesión
|   |   |-- login_screen.dart    # Inicio de sesión
|   |   |-- register_screen.dart # Registro y perfil inicial
|   |   |-- home_screen.dart     # Feed vertical de convocatorias
|   |   |-- grupos_screen.dart   # Grupos de una convocatoria
|   |   |-- crear_grupo_screen.dart # Alta de equipo y déficit
|   |   |-- mis_grupos_screen.dart # Grupos que lidera el usuario
|   |   |-- notificaciones_screen.dart # Bandeja de invitaciones
|   |   |-- profile_screen.dart  # Perfil propio
|   |   |-- editar_perfil_screen.dart # Edición de perfil
|   |   |-- crear_convocatoria_screen.dart # Alta/edición de eventos (admin)
|   |   `-- admin_panel_screen.dart # Panel de administración (4 pestañas)
|   |-- theme/                   # Sistema de diseño institucional
|   |   |-- app_colors.dart      # AppColors, AppGradients, TipoConvocatoria
|   |   |-- app_theme.dart       # AppTheme.light() (Material 3)
|   |   `-- app_typography.dart  # Escala tipográfica (google_fonts)
|   `-- widgets/                 # Componentes reutilizables
|       |-- card_convocatoria.dart
|       |-- gradient_button.dart
|       |-- hub_glyph.dart       # Isotipo de la marca
```

```

|         |-- menu_widget.dart           # Navegación inferior
|         |-- selector_habilidades.dart  # Selector/creador de habilidades
|         `-- selector_usuario.dart      # Buscador del directorio
|-- cloudflare/
|   |-- upload-worker.js                 # Worker de subida y entrega (R2)
|   `-- wrangler.toml
|-- firestore.rules                       # Reglas de seguridad del servidor
|-- test/                                # Pruebas automatizadas
`-- docs/                                 # Modelo de datos e informe

```

3.4 Dependencias principales

Tabla 4: Dependencias externas y su función

Paquete	Uso en el proyecto
firebase_core, firebase_auth, cloud_firestore	Inicialización, autenticación y base de datos.
shared_preferences	Caché local de la sesión y del perfil.
google_fonts	Familias tipográficas del sistema de diseño.
flutter_animate	Animaciones de entrada de las pantallas.
video_player	Reproducción del video promocional en el feed.
palette_generator	Extracción del color dominante de la imagen del evento para componer el fondo del feed.
image_picker, http	Selección y envío de multimedia al Worker de Cloudflare.
fake_cloud_firestore, firebase_auth_mock	Dobles de prueba de Firebase (solo desarrollo).
flutter_lints	Conjunto de reglas del analizador estático.
flutter_launcher_icons	Generación de íconos de Android y <i>favicon</i> web.

4 Modelo de Datos

4.1 Colecciones en Cloud Firestore

El modelo se implementó sobre seis colecciones raíz. Las claves de los documentos usan *snake_case* (por ejemplo `escuela_profesional`), y el identificador del documento de perfil es el propio `uid` de Firebase Authentication, lo que enlaza identidad y perfil sin una tabla intermedia.

Tabla 5: Colecciones de Cloud Firestore

Colección	Id del documento	Contenido y campos relevantes
usuarios	uid de Auth	nombre, correo, habilidades[], y opcionalmente <code>escuela_profesional</code> y <code>semestre</code> (estudiantes) u <code>ocupacion</code> (externos). Es el directorio que consume el motor de emparejamiento.
convocatorias	autogenerado	titulo, descripcion, tipo, fecha_publicacion, fecha_cierre, premio, requisitos, y opcionalmente <code>imagen_url</code> y <code>video_url</code> .
grupos	autogenerado	convocatoria_id, lider_id, lider_nombre, miembros[] (desnormalizados), capacidad_max, habilidades_buscadadas[], abierto, estado (borrador o publicado).
invitaciones	autogenerado	grupo_id, grupo_nombre, lider_id, usuario_id, usuario (perfil embebido), puntuacion, habilidades_coincidentes[], estado, fecha.
solicitudes	autogenerado	grupo_id, lider_id, usuario_id, usuario (perfil embebido), estado, fecha.
admins	uid de Auth	nombre, correo, rol (admin o admin_paqarina), fundador. La existencia del documento <i>es</i> el rol.

4.2 Estrategia de desnormalización

Firestore no dispone de *joins*, de modo que el diseño privilegia la lectura sobre la normalización:

- **MiembroResumen embebido en el grupo.** El documento del grupo no guarda referencias a los perfiles de sus integrantes, sino un resumen (`id`, `nombre`, `escuela_profesional`, `semestre`) suficiente para pintar la tarjeta del integrante. Así, abrir un grupo con ocho miembros cuesta **una sola lectura** en lugar de nueve (RNF-03). El perfil completo y autoritativo sigue viviendo en `usuarios/{uid}`.

- **Perfil e instantánea de compatibilidad embebidos en la invitación.** Cada invitación guarda el perfil del candidato y las habilidades coincidentes *en el momento del cálculo*. La bandeja de notificaciones puede así explicar al usuario por qué fue invitado, aunque su perfil haya cambiado después.
- **Escalares redundantes para las reglas.** Invitaciones y solicitudes duplican `usuario_id` y `lider_id` como campos planos, porque las *Security Rules* y las consultas de Firestore no pueden filtrar por campos anidados dentro de un mapa.

4.3 Estrategia de consultas

Todas las consultas usan **un solo campo en el where** y aplican el resto de los filtros en memoria. Por ejemplo, para obtener las invitaciones pendientes de un usuario se consulta `where('usuario_id', isEqualTo: uid)` y se filtra el estado `pendiente` en el cliente, en lugar de encadenar dos condiciones. La razón es concreta: Firestore exige un **índice compuesto** declarado y desplegado para cada consulta multicampo, y el volumen de datos del piloto no justifica esa complejidad operativa. El archivo `firestore.indexes.json` permanece, por diseño, vacío.

4.4 Modelo lógico relacional

La figura 1 presenta el mismo dominio expresado en forma relacional normalizada, útil como referencia conceptual y como guía para una eventual migración a una base de datos relacional. Las listas embebidas del modelo de documentos (`habilidades`, `miembros`, `habilidades_coincidentes`) aparecen allí normalizadas como tablas intermedias N:M.

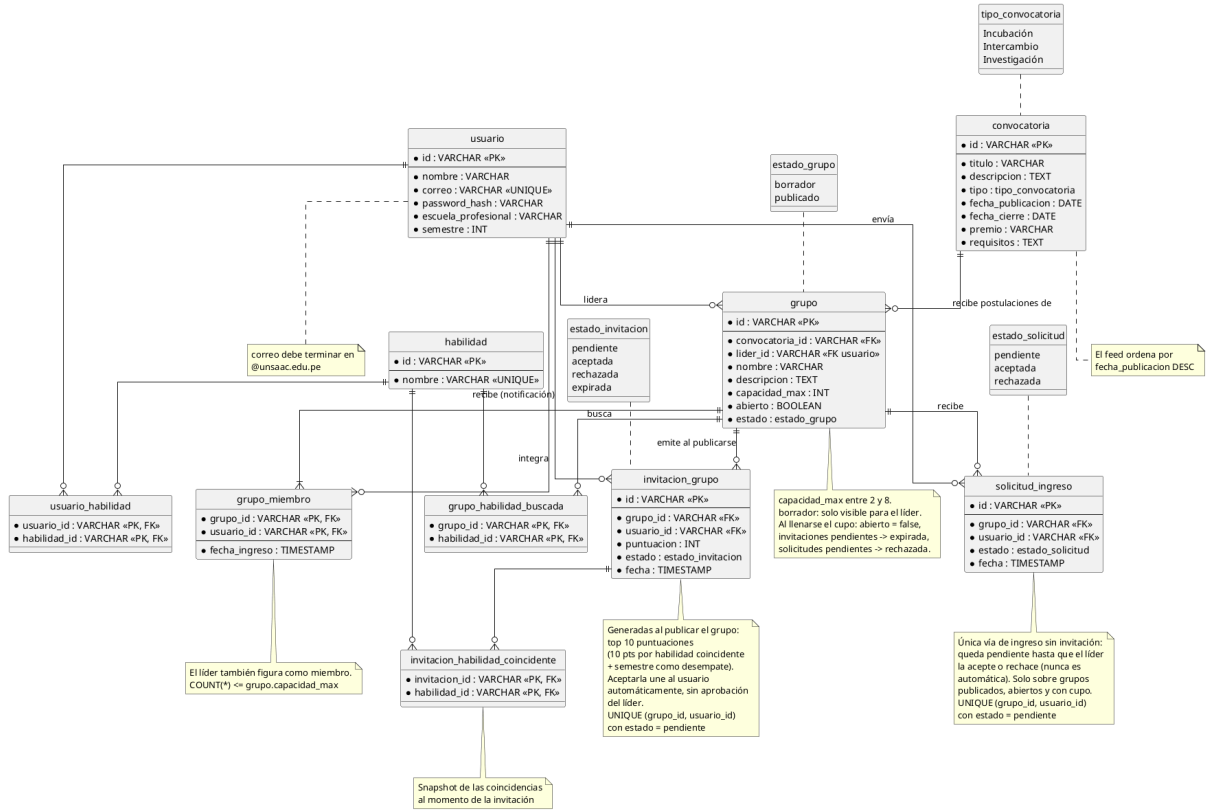


Figura 1: Modelo lógico relacional del dominio de Yuyanna Hub

5 Motor de Emparejamiento Automático

5.1 Modelo conceptual

El emparejamiento se modela como el cruce entre la **oferta de competencias** de un estudiante y la **demanda de competencias** de un equipo en formación. Cada usuario u posee un conjunto de habilidades etiquetadas $H_u = \{h_1, h_2, \dots, h_n\}$; cada grupo g declara un conjunto de habilidades buscadas B_g , que representa explícitamente su déficit.

A diferencia de un emparejamiento simétrico entre personas, aquí el objeto emparejado es el **equipo**, y el evento que dispara el cálculo es la **publicación del grupo**: mientras el grupo está en estado **borrador** es privado y no genera ninguna notificación; al publicarse, el sistema busca activamente a quienes lo completen.

5.2 Función de compatibilidad implementada

Sea $M(u, g)$ el conjunto de habilidades buscadas por el grupo que el usuario efectivamente cubre:

$$M(u, g) = \{b \in B_g \mid \exists h \in H_u : b \approx h\}$$

donde $\approx$ denota la relación de coincidencia difusa definida en la sección 5.3. El puntaje de compatibilidad es entonces:

$$\text{Score}(u, g) = 10 \cdot \underbrace{|M(u, g)|}_{\text{habilidades cubiertas}} + \underbrace{\text{mín}(\text{sem}(u), 9)}_{\text{criterio de desempate}}$$

siendo $\text{sem}(u)$ el semestre declarado por el usuario (0 si no lo declaró, por ejemplo un perfil externo).

Propiedad de ordenamiento. La constante 10 y el recorte del semestre a 9 no son arbitrarios: garantizan que el puntaje ordene a los candidatos **lexicográficamente** por el par $(|M(u, g)|, \text{sem}(u))$.

Demostración. El término de desempate está acotado por $0 \leq \min(\text{sem}(u), 9) \leq 9 < 10$. Por lo tanto, si $|M(u_1, g)| > |M(u_2, g)|$, entonces $|M(u_1, g)| \geq |M(u_2, g)| + 1$ y

$$\text{Score}(u_1, g) \geq 10|M(u_2, g)| + 10 > 10|M(u'_1, g)| + 9 \geq \text{Score}(u_2, g),$$

de modo que ninguna diferencia de semestre puede revertir una diferencia en el número de habilidades cubiertas. El semestre solo decide entre candidatos empatados en cobertura. $\square$

La consecuencia de diseño es intencional: **la cobertura del déficit manda; la antigüedad académica solo desempata**. Un estudiante de segundo semestre que cubre tres habilidades buscadas siempre se ordena por encima de uno de décimo que cubre dos.

5.3 Coincidencia difusa de habilidades

Las habilidades son texto libre escrito por personas distintas, de modo que la igualdad exacta de cadenas produciría falsos negativos evidentes (“Python” frente a “python”, “Diseño UX” frente a “diseno ux”). La relación $b \approx h$ se define en tres pasos:

1. **Normalización:** ambas cadenas se recortan, se pasan a minúsculas y se les eliminan las tildes ($\acute{a} \rightarrow a$, $\acute{e} \rightarrow e$, $\acute{i} \rightarrow i$, $\acute{o} \rightarrow o$, $\acute{u}/\ddot{u} \rightarrow u$).
2. **Igualdad:** si las cadenas normalizadas son idénticas, coinciden.
3. **Contención con umbral:** si una contiene a la otra *y* la cadena contenida tiene al menos 4 caracteres, también coinciden. Así “Flutter” coincide con “Flutter avanzado”, mientras que el umbral de 4 caracteres evita que fragmentos como “UX” o “IA” disparen coincidencias espurias dentro de palabras no relacionadas.

5.4 Algoritmo de publicación y notificación

El procedimiento completo, implementado en `GrupoService.publicarGrupo`, es el siguiente:

Entrada : grupo g (en estado borrador)

Salida : g publicado + lista de invitaciones emitidas

1. $g.\text{estado} \leftarrow \text{publicado}$; persistir g
2. si $B_g = \text{vacío}$ o g está lleno : retornar sin invitaciones
3. $C \leftarrow \text{directorio completo de usuarios} \setminus \text{miembros actuales de } g$
4. $P \leftarrow \text{vacío}$
5. para cada candidato u en C :
6. $M \leftarrow \text{habilidades de } B_g \text{ cubiertas por } u$ (coincidencia difusa)
7. si $M = \text{vacío}$: continuar # no se notifica a quien no aporta
8. $P \leftarrow P + (u, 10 * |M| + \min(\text{sem}(u), 9), M)$

9. ordenar P por puntaje descendente
10. para cada uno de los primeros 10 elementos de P:
11. crear invitación pendiente con puntaje y coincidencias

Tres decisiones merecen destacarse:

- **Filtro estricto de relevancia (línea 7).** Un candidato sin ninguna habilidad coincidente *nunca* es notificado, sin importar cuántos huecos tenga el grupo. Esto cumple el criterio de “umbral mínimo” del diseño original: la bandeja de invitaciones no se contamina con ruido.
- **Tope de diez invitaciones (GrupoService.maxInvitaciones).** Acota tanto el costo de escritura en Firestore como la presión social sobre el directorio: publicar un grupo no genera una notificación masiva al campus.
- **Instantánea del cálculo.** El puntaje y las habilidades coincidentes se guardan en la invitación, no se recalculan al leerla.

Complejidad. El cálculo es $O(|C| \cdot |B_g| \cdot |H_u|)$ comparaciones de cadenas cortas, ejecutado una única vez por publicación de grupo y sobre un directorio del orden de los miles de perfiles: perfectamente asumible en el cliente. Su costo dominante no es el cómputo sino la lectura del directorio (una consulta) y la escritura de hasta diez invitaciones.

5.5 Ciclo de vida del ingreso a un equipo

El emparejamiento produce invitaciones, pero el ingreso efectivo admite dos vías, deliberadamente asimétricas:

Tabla 6: Vías de ingreso a un grupo

Vía	Quién la origina	Quién decide
Invitación	El sistema, al publicarse el grupo	El invitado: aceptar lo une de inmediato, porque la invitación ya lo pre-aprobó.
Solicitud de ingreso	El propio usuario, desde la vista de grupos	El líder: acepta o rechaza; nunca es automática.
Alta directa	El líder, desde su grupo	El líder, eligiendo del directorio.

Al completarse el cupo (`integrantes = capacidad_max`) se dispara un **cierre en cascada**: el grupo pasa a `abierto = false`, todas sus invitaciones pendientes se marcan **expirada** y todas sus solicitudes pendientes se marcan **rechazada**. Simétricamente, cuando un usuario entra al grupo por una vía, sus pendientes por la otra vía se resuelven para que no queden notificaciones huérfanas. Esta cascada es la razón técnica del compromiso de seguridad descrito en la sección 6.5.

6 Seguridad y Control de Acceso

6.1 Autenticación

La autenticación se delega íntegramente a **Firestore Authentication** con proveedor de correo y contraseña. La aplicación *nunca* recibe, transporta ni almacena contraseñas: Firestore gestiona el almacenamiento con derivación de clave (*script*) y expone únicamente el `uid`. Al registrarse, el usuario recibe un correo de verificación; existe además el flujo de restablecimiento de contraseña desde la pantalla de inicio de sesión.

El perfil se escribe en `usuarios/{uid}` y se cachea localmente en `LocalSessionManager (shared_preferences)`, de modo que el arranque de la aplicación resuelve la sesión sin una lectura remota. La pantalla de bienvenida (`SplashScreen`) consulta esa bandera local tras 2.5 segundos y deriva al feed o al inicio de sesión.

6.2 Identidad institucional

La constante `Usuario.dominioUnsaac (@unsaac.edu.pe)` y la propiedad derivada `esUnsaac` determinan si una cuenta pertenece a la comunidad universitaria, lo que se refleja en el perfil con la insignia *Estudiante UNSAAC*. Se trata de una **distinción, no de una restricción**: la justificación se detalla en la sección 10.

6.3 Roles de administración

Los roles viven en la colección `admins`, donde *la existencia del documento es el rol*:

- **admin** — administrador completo. Accede al panel de administración y puede crear, editar y eliminar eventos, ver el directorio y los grupos, y promover o degradar a otros administradores.
- **admin_paqarina** — gestor de la incubadora. Puede publicar eventos, pero no accede al panel ni edita o elimina lo publicado por otros.

El **problema del primer administrador** (nadie puede promover al primero si aún no hay ninguno) se resuelve con un *bootstrap* por lista de correos fundadores: si un usuario cuyo correo figura en `AdminService.correosFundadores` inicia sesión y todavía no tiene documento en `admins`, la aplicación se lo crea con rol **admin** al cargar la pantalla principal. La misma lista está replicada en la función `esFundador()` de `firestore.rules`; ambas **deben mantenerse sincronizadas**, pues la regla del servidor es la que realmente autoriza la operación.

6.4 Reglas de seguridad del servidor

La autorización no se confía al cliente. `firestore.rules` define las funciones `estaAutenticado()`, `esAdmin()`, `tieneRolAdmin()` y `esFundador()`, y con ellas:

6.5 Compromiso documentado sobre la escritura de equipos

Las tres últimas filas de la tabla anterior son deliberadamente permisivas y constituyen el principal compromiso técnico del proyecto. La causa es estructural: **sin Cloud**

Tabla 7: Permisos por colección según las Security Rules

Colección	Lectura	Escritura
<code>usuarios</code>	Autenticado	Solo el propio usuario sobre su documento (<code>uid</code> coincidente). Borrado prohibido.
<code>convocatorias</code>	Autenticado	Crear: cualquier rol admin. Editar y eliminar: solo admin completo.
<code>admins</code>	Admin, o el propio interesado	Crear: un fundador sobre sí mismo, o un admin sobre otros. Cambiar rol y eliminar: solo admin.
<code>grupos</code>	Autenticado	Crear: solo si <code>lider_id</code> es el propio <code>uid</code> . Eliminar: líder o admin. Actualizar: cualquier autenticado.
<code>invitaciones, solicitudes</code>	Autenticado	Cualquier autenticado.

Functions (no disponibles en el plan gratuito *Spark*), la lógica de equipos corre en el cliente, y esa lógica **toca documentos de otros usuarios**. Cuando un estudiante acepta la última plaza de un grupo, su dispositivo debe expirar las invitaciones pendientes *de terceros* y rechazar las solicitudes *de terceros*. Ninguna regla que restrinja la escritura a “dueño o líder” permitiría esa cascada.

Se optó por hacer el compromiso **explícito y acotado** en lugar de disimularlo: está comentado en el propio archivo de reglas, se limita a tres colecciones que no contienen credenciales ni datos sensibles, y todas las operaciones siguen exigiendo usuario autenticado. La ruta de remediación es conocida y está descrita en la sección 11: migrar la cascada a Cloud Functions bajo el plan *Blaze* y endurecer las reglas a “dueño o líder”, ejecutando el cierre dentro de una transacción.

6.6 Gestión de multimedia

Firebase Storage tampoco está disponible en el plan gratuito, por lo que el material audiovisual de los eventos se resolvió con un **Cloudflare Worker** propio sobre un *bucket* R2:

- **POST /upload** valida una cabecera **X-Upload-Token**, sanea el nombre del archivo (solo caracteres alfanuméricos, punto, guion y guion bajo; máximo 80 caracteres), le antepone marca de tiempo y un UUID parcial para evitar colisiones, lo guarda en R2 y devuelve la URL pública.
- **GET /file/<clave>** entrega el archivo con **Cache-Control** inmutable de un año y **soporte del encabezado Range** (respuestas 206), requisito para que el video del feed pueda buscarse y reproducirse progresivamente.

En Firestore solo se persiste la URL (`imagen_url`, `video_url`); el binario nunca entra en la base de datos.

7 Diseño de Interfaz y Flujo de Navegación

7.1 Sistema de diseño

Todo el estilo está centralizado en `lib/theme/`, y ninguna pantalla codifica colores ni estilos de texto directamente. La paleta es institucional (rojo, azul y dorado de la UNSAAC) sobre un fondo de papel cálido:

Tabla 8: Paleta institucional (extracto de `AppColors`)

Rol	Valor	Uso
<code>paper</code>	<code>#FAF7F2</code>	Fondo general de la aplicación.
<code>ink / inkSecondary</code>	<code>#1F1A2E / #6B6478</code>	Texto principal y secundario.
<code>red / redFill / redDeep</code>	<code>#A6213B ...</code>	Acción primaria, eventos de investigación.
<code>blue / blueFill / blueDeep</code>	<code>#1F4E8C ...</code>	Hackatones e intercambio.
<code>gold / goldFill / goldDeep</code>	<code>#A87410 ...</code>	Incubación y preincubación.

La clase auxiliar `TipoConvocatoria` traduce el tipo textual de un evento a su color y su degradado, de modo que el código cromático es consistente en el feed, las tarjetas y las etiquetas. El tema es Material 3 en variante clara únicamente, con tipografía servida por `google_fonts` y animaciones de entrada con `flutter_animate`.

7.2 Flujo de navegación

La navegación es explícita y sin rutas nombradas:

```
SplashScreen (2.5 s, lee la sesión local)
  |-- sin sesión --> LoginScreen <--> RegisterScreen
  `-- con sesión --> HomeScreen
                        |
                        |-- IndexedStack de 3 pestañas (MenuWidget inferior)
                        |   |-- 0. Feed vertical de convocatorias
                        |   |-- 1. Panel de administración (solo admin)
                        |   `-- 2. Perfil --> EditarPerfilScreen
                        |
                        |-- NotificacionesScreen (campana + contador)
                        |-- CrearConvocatoriaScreen (solo roles admin)
                        `-- GruposScreen (de una convocatoria)
                              |-- CrearGrupoScreen
                              `-- MisGruposScreen
```

La pantalla principal mantiene las tres pestañas vivas en un `IndexedStack` (el estado del feed no se pierde al visitar el perfil) y empuja las pantallas de detalle con `Navigator.push`. La pestaña de administración solo aparece para el rol `admin` completo.

7.3 El feed de convocatorias

La decisión de interfaz más característica del proyecto es presentar las convocatorias como un **PageView vertical a pantalla completa**, con el lenguaje visual de las plataformas de video corto que el público objetivo ya domina, en lugar de una lista de tarjetas administrativas. Cada página incluye:

- La imagen o el video del evento como fondo. La imagen se ajusta al ancho completo sin recortar los lados, y el espacio sobrante se rellena con un degradado derivado del **color dominante de la propia imagen**, calculado con `palette_generator`; mientras se calcula, se muestra el degradado del tipo de evento.
- Título, tipo, fecha de cierre, premio y requisitos, con la descripción truncada y un enlace “Leer más” que solo aparece si el texto no cabe completo.
- Un riel lateral de acciones y el acceso directo a los grupos de esa convocatoria.
- Un **modo inmersivo**: el doble toque oculta todos los elementos superpuestos y deja únicamente la imagen; otro doble toque los restituye.

La barra superior muestra el contador de invitaciones pendientes, obtenido mediante `getInvitacionesParaUsuario`, que actúa como notificación interna a falta de mensajería *push*.

7.4 Pantallas de formación de equipos

- `CrearGrupoScreen` — recibe la convocatoria ya seleccionada y solicita nombre, descripción, capacidad máxima (validada entre 2 y 8 integrantes), integrantes iniciales tomados del directorio mediante `selector_usuario`, y las habilidades buscadas mediante `selector_habilidades`. El grupo nace en estado **borrador**.
- `MisGruposScreen` — lista los grupos que el usuario lidera, distingue borradores de publicados y ofrece la acción de publicar, que es la que dispara el motor de emparejamiento.
- `GruposScreen` — muestra los grupos publicados de una convocatoria (más los borradores propios), permite solicitar el ingreso y, al líder, resolver las solicitudes recibidas.
- `NotificacionesScreen` — bandeja de invitaciones pendientes; cada tarjeta indica el grupo, el puntaje y *qué habilidades* motivaron la invitación, con las acciones de unirse o rechazar.

7.5 Panel de administración

`AdminPanelScreen` organiza la gestión en cuatro pestañas: **Eventos** (alta, edición y eliminación de convocatorias), **Usuarios** (directorio con la etiqueta de rol de cada uno), **Grupos** (todos los grupos creados, con eliminación) y **Admins** (promoción por correo, cambio de rol `admin` ↔ `admin_paqarina` y revocación).

7.6 Capturas de pantalla de la aplicación

A continuación, se presentan las capturas de pantalla de las principales interfaces de la aplicación Yuyanna Hub.

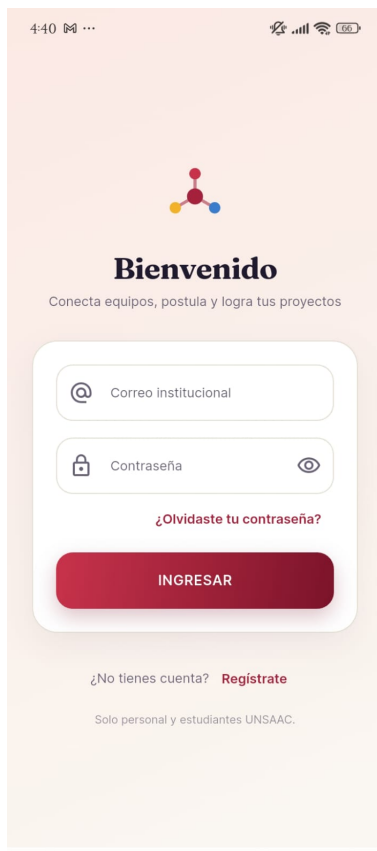


Figura 2: Inicio de sesión

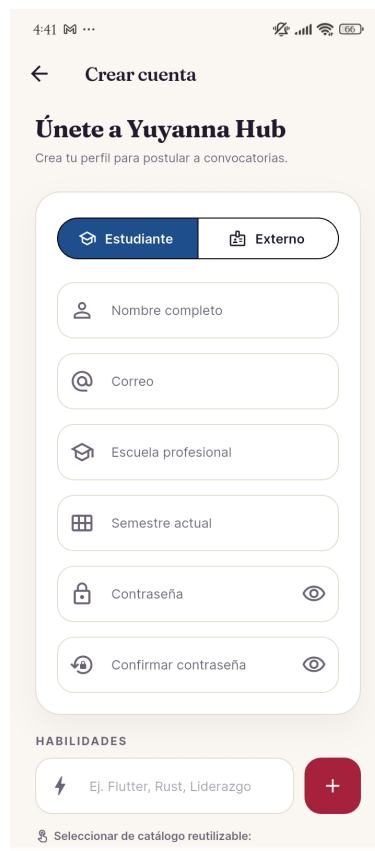


Figura 3: Registro



Figura 4: Feed de convocatorias



Figura 5: Grupos



Figura 6: Crear grupo

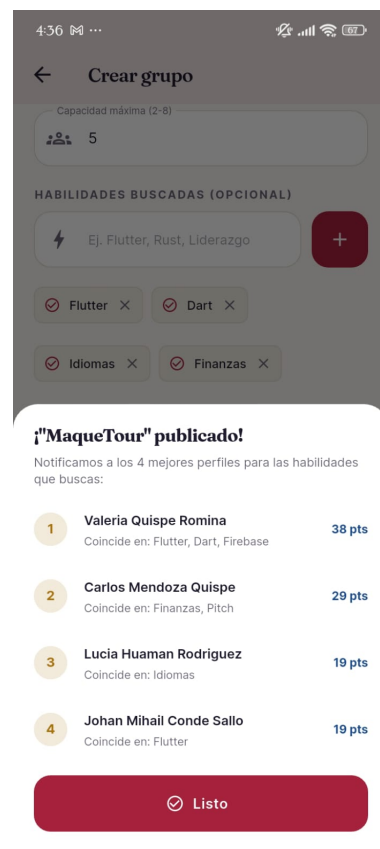


Figura 7: Emparejamiento

8 Pruebas y Verificación

8.1 Estrategia

La estrategia de pruebas concentra el esfuerzo donde está el riesgo: la **lógica de dominio** de formación de equipos y emparejamiento, que es la única parte del sistema con reglas de negocio propias. Las pruebas se ejecutan contra `fake_cloud_firestore`, un doble en memoria de Firestore, de modo que verifican el comportamiento real del servicio —incluidas sus consultas y escrituras— sin tocar la base de datos de producción ni requerir conexión.

8.2 Casos de prueba automatizados

Tabla 9: Suite de pruebas automatizadas

Archivo	Caso verificado
<code>grupo_service_test</code>	El grupo se crea en estado borrador , con el líder registrado como miembro y los integrantes iniciales incorporados.

Archivo	Caso verificado
<code>grupo_service_test</code>	Al publicar, solo se notifica a perfiles con al menos una habilidad coincidente; las invitaciones salen ordenadas por puntaje descendente y no superan el máximo de diez. El perfil sin coincidencias no recibe invitación.
<code>grupo_service_test</code>	Un invitado que acepta se incorpora automáticamente al grupo, sin intervención del líder.
<code>grupo_service_test</code>	Una solicitud de ingreso permanece pendiente hasta que el líder la resuelve; nunca produce un ingreso automático.
<code>grupo_service_test</code>	Al llenarse el cupo, el grupo se cierra y las invitaciones y solicitudes pendientes quedan expiradas o rechazadas (cierre en cascada).
<code>habilidades_test</code>	El catálogo de habilidades no está vacío; agregarHabilidad inserta nuevas entradas y evita duplicados sin distinguir mayúsculas; el texto separado por comas se procesa elemento por elemento; la búsqueda filtra correctamente.
<code>habilidades_test</code>	El <i>widget SelectorHabilidades</i> renderiza las habilidades seleccionadas como <i>chips</i> y las elimina una a una al tocar su ícono de cierre.
<code>widget_test</code>	Prueba de humo de arranque: la pantalla de bienvenida muestra el nombre de la aplicación (adelantando el temporizador de 2.5 s para no dejar temporizadores pendientes).

8.3 Resultados de la verificación

Tabla 10: Resultados de las herramientas de verificación

Herramienta	Comando	Resultado
Analizador estático	<code>flutter analyze</code>	Sin observaciones (<i>No issues found</i>).
Pruebas automatizadas	<code>flutter test</code>	12 de 12 pruebas superadas.

8.4 Métricas del producto

8.5 Verificación manual

Además de la suite automatizada, se validaron manualmente los recorridos que dependen de servicios externos y no son reproducibles con dobles: registro con verificación de correo, restablecimiento de contraseña, *bootstrap* del administrador fundador, publicación de un evento con imagen y video (subida al Worker de Cloudflare y reproducción con búsqueda dentro del video), y el ciclo completo de publicación de grupo con recepción de la invitación en una segunda cuenta.

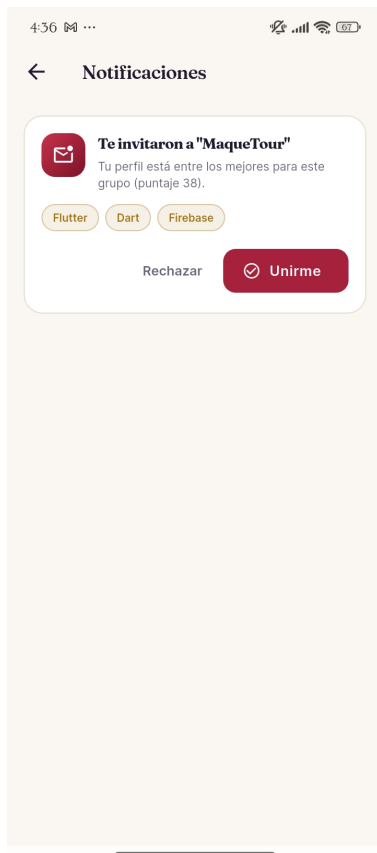


Figura 8: Invitaciones



Figura 9: Perfil estudiante



Figura 10: Editar perfil

Tabla 11: Métricas del código fuente

Métrica	Valor
Archivos fuente Dart (<code>lib/</code> y <code>test/</code>)	39
Líneas de código Dart	$\approx 9\,770$
Modelos de dominio	6
Servicios	6
Pantallas	12
<i>Widgets</i> reutilizables	6
Colecciones de Firestore	6
Pruebas automatizadas	12
Plataformas objetivo	Android y web



Figura 11: Panel Admin

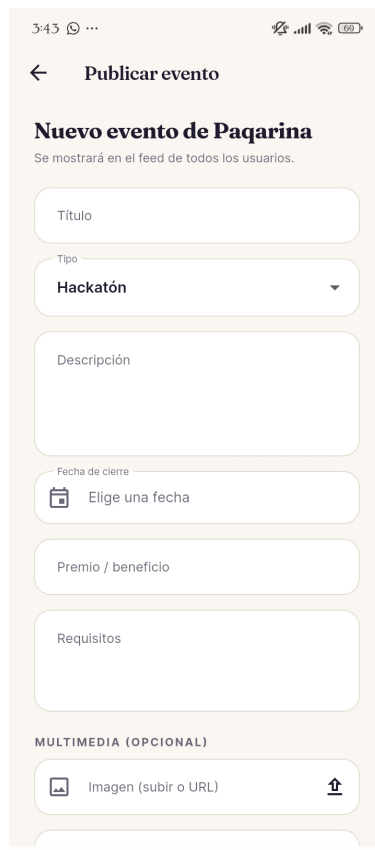


Figura 12: Publicar Evento

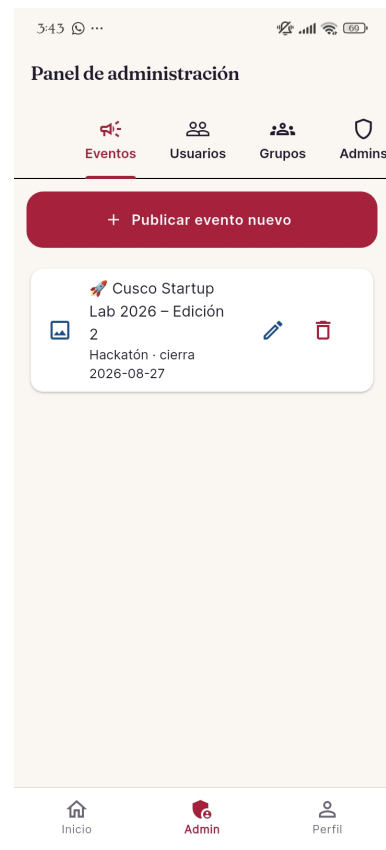


Figura 13: Superadmin

9 Trazabilidad: Planificado frente a Alcanzado

Tabla 12: Cumplimiento de los requerimientos funcionales

Código	Requerimiento	Estado	Observación
RF-01	Autenticación de cuentas	Parcial	Implementada con correo y contraseña, verificación de correo y restablecimiento. Se sustituyó el código OTP planificado y se relajó la restricción de dominio (véase 9.1).
RF-02	Perfil de habilidades	Implementado	Incluye tipo de perfil estudiante/externo. No se implementó el enlace a portafolios externos.

Código	Requerimiento	Estado	Observación
RF-03	Mapeo de convocatorias	Implementado	Alta, edición y eliminación desde el panel, con imagen y video. Los tipos activos son Hackatón, Incubación y Pre incubación.
RF-04	Publicación de equipos y déficit	Implementado	El déficit se declara como conjunto de habilidades buscadas, no como plazas nominadas con rol y escuela.
RF-05	Motor de emparejamiento	Parcial	Implementada la dirección <i>equipo</i> → <i>candidatos</i> (invitaciones automáticas al publicar). La recomendación inversa <i>estudiante</i> → <i>equipos</i> quedó pendiente.
RF-06	Postulación y aprobación	Implementado	Ambas vías (invitación y solicitud) más el cierre automático en cascada.
RF-07	Bandeja de notificaciones	Parcial	Bandeja interna con contador, funcional. No hay notificaciones <i>push</i> (requieren FCM y plan de pago).
RF-08	Panel de administración	Implementado	Cuatro pestañas con dos roles diferenciados.
RF-09	Gestión de multimedia	Implementado	Worker de Cloudflare sobre R2, con soporte de Range para video.

Tabla 13: Cumplimiento de los requerimientos no funcionales

Código	Requerimiento	Estado	Observación
RNF-01	Multiplataforma	Implementado	Android y web desde una sola base de código. iOS quedó fuera del alcance (requiere equipo y cuenta de desarrollador Apple).
RNF-02	Costo de operación	Implementado	Opera íntegramente sobre planes gratuitos.

Código	Requerimiento	Estado	Observación
RNF-03	Eficiencia de lectura	Implementado	Miembros desnormalizados (una lectura por grupo) y consultas de un solo campo (cero índices compuestos).
RNF-04	Seguridad y control de acceso	Parcial	Reglas del servidor activas y roles reforzados, con el compromiso explícito de escritura sobre grupos, invitaciones y solicitudes (6.5).
RNF-05	Persistencia de sesión	Implementado	Caché local del perfil en <code>shared_preferences</code> .
RNF-06	Idioma y consistencia	Implementado	Dominio, interfaz e identificadores en español.
RNF-07	Calidad de código	Implementado	<code>flutter analyze</code> sin observaciones; 12 pruebas en verde.

9.1 Desviaciones respecto del plan original y su justificación

1. **Autenticación por contraseña en lugar de código OTP.** El envío de códigos de un solo uso exige una función de servidor que genere, almacene y valide el código, además de un proveedor de correo transaccional; ambas cosas están fuera del plan gratuito. Firebase Authentication ofrece, sin costo, un flujo equivalente en garantías de posesión del buzón (verificación de correo y restablecimiento por enlace firmado) con una implementación notablemente más simple y auditada.
2. **El dominio institucional distingue, no restringe.** El plan original bloqueaba el acceso a cuentas ajenas a `@unsaac.edu.pe`. Durante el desarrollo se identificó que los usuarios secundarios legítimos —mentores externos, egresados sin correo institucional vigente y los propios gestores de la incubadora— quedaban excluidos por esa regla. La solución adoptada conserva la señal de confianza sin perder cobertura: cualquiera puede registrarse, pero solo las cuentas del dominio exhiben la insignia *Estudiante UNSAAC*, y el modelo de perfil admite explícitamente el tipo *externo* con su campo de ocupación.
3. **Backend gestionado en lugar de API propia en Rust/Axum.** El plan contemplaba un servicio propio. Se optó por Firebase para eliminar el costo de operación y de despliegue de un servidor durante el piloto, y para concentrar el esfuerzo del semestre en el dominio (emparejamiento y formación de equipos) en lugar de en infraestructura. El costo de esta decisión está documentado en 6.5.
4. **Función de compatibilidad aditiva en lugar de similitud de Jaccard ponderada.** La formulación inicial combinaba similitud de Jaccard, coincidencia de escuela y un término de diversidad, con pesos α, β, γ . Se reemplazó por la función

descrita en la sección 5 por tres razones concretas: (a) el denominador de Jaccard $|H_u \cup H_v|$ **penaliza a los perfiles más completos** —un estudiante que declara veinte habilidades obtiene menor similitud que uno que declara tres, aun cubriendo el mismo déficit—, lo que es exactamente el comportamiento contrario al deseado; (b) los pesos α, β, γ no eran calibrables sin datos históricos de uso, de los que el piloto aún carece; y (c) la función adoptada es *explicable*: la invitación puede mostrar literalmente qué habilidades motivaron el puntaje, lo que la formulación fraccionaria no permite comunicar con claridad. El término de diversidad multidisciplinaria quedó registrado como trabajo futuro (11).

5. **Déficit expresado como habilidades y no como plazas nominadas.** Modelar cada vacante con rol, escuela deseada y habilidades habría multiplicado la complejidad del formulario de creación de grupo, que es el punto de mayor fricción del recorrido. Se optó por un único conjunto de habilidades buscadas a nivel de equipo, que preserva la capacidad de emparejar sin exigir al líder un diseño organizacional previo.

10 Limitaciones Conocidas

Se documentan de manera explícita las limitaciones del estado actual, por honestidad técnica y para orientar el trabajo posterior:

1. **Escritura permisiva en tres colecciones.** Cualquier usuario autenticado puede escribir en `grupos`, `invitaciones` y `solicitudes`. Es el compromiso analizado en 6.5, aceptable para un piloto controlado e inaceptable para una operación abierta y prolongada.
2. **Ausencia de transacciones.** El cierre en cascada al llenarse un grupo se ejecuta como una secuencia de escrituras independientes desde el cliente. Dos usuarios que acepten la última plaza simultáneamente podrían, en el peor caso, exceder la capacidad declarada. La solución correcta es una transacción de Firestore o, mejor, una Cloud Function.
3. **Token de subida embebido en el cliente.** `MediaConfig.uploadToken` viaja dentro de la aplicación compilada y, por tanto, es extraíble. Mitiga el abuso casual, no el dirigido. La corrección consiste en que el Worker valide el *ID token* de Firebase del usuario y verifique su rol de administrador, en lugar de un secreto compartido estático.
4. **Lista de fundadores duplicada.** Los correos fundadores están escritos tanto en `AdminService` como en `firestore.rules`; una divergencia entre ambos rompería el *bootstrap* del primer administrador de forma silenciosa.
5. **Emparejamiento como evento puntual.** El cálculo ocurre una sola vez, al publicar el grupo. Un usuario que se registra o que amplía sus habilidades después de esa publicación no será considerado, aunque sea el candidato ideal.
6. **Directorio completo en memoria.** `getUsuarios()` descarga todos los perfiles para puntuarlos. Es adecuado para el orden de magnitud del piloto, pero no escala a decenas de miles de usuarios ni al presupuesto de lecturas del plan gratuito.
7. **Sin notificaciones *push*.** El contador de la barra superior solo se actualiza cuando la aplicación está abierta; una invitación puede pasar inadvertida hasta el siguiente ingreso.
8. **Habilidades como texto libre.** El catálogo es un conjunto en memoria que cada instalación amplía localmente; no se sincroniza entre dispositivos, de modo que la coincidencia difusa carga con el peso de las variantes ortográficas.
9. **Nombre engañoso de un archivo.** `lib/database/sqlite_helper.dart` no contiene SQLite, sino el gestor de sesión sobre `shared_preferences`; el nombre sobrevive de una etapa anterior del diseño.

11 Trabajo Futuro

1. **Migrar la lógica de equipos a Cloud Functions (plan *Blaze*)**. Es la mejora de mayor impacto: permitiría endurecer las reglas a “dueño o líder”, ejecutar el cierre en cascada dentro de una transacción y eliminar simultáneamente las limitaciones 1 y 2.
2. **Recomendación inversa (estudiante $\rightarrow$ equipos)**. Completar el RF-05 con la vista “equipos que te necesitan”, reutilizando la misma función de compatibilidad recorrida en sentido contrario sobre los grupos publicados y abiertos de las convocatorias vigentes.
3. **Reemparejamiento incremental**. Recalcular las invitaciones cuando un usuario nuevo se registra o actualiza sus habilidades, mientras existan grupos publicados con plazas y déficit sin cubrir.
4. **Incorporar el término de diversidad multidisciplinaria**. Añadir una bonificación por escuela profesional aún no representada en el equipo, respetando la propiedad de ordenamiento lexicográfico (un peso estrictamente menor que el paso de 10 puntos por habilidad), de manera que la diversidad desempate sin desplazar a la cobertura del déficit.
5. **Notificaciones *push* con Firebase Cloud Messaging**, para que una invitación llegue al dispositivo aunque la aplicación esté cerrada.
6. **Catálogo de habilidades compartido en Firestore**, con normalización y fusión de sinónimos administrada desde el panel, reduciendo la dependencia de la coincidencia difusa.
7. **Autorización real del Worker de subida** mediante verificación del *ID token* de Firebase y del rol de administrador.
8. **Métricas de la incubadora**. Instrumentar tasa de aceptación de invitaciones, tiempo medio hasta completar un equipo y diversidad de escuelas por equipo formado: son precisamente los datos que hoy faltan para calibrar empíricamente los pesos de la función de compatibilidad, y los que permitirían a Paqarina Wasi medir el efecto de la herramienta sobre la calidad de las postulaciones.

12 Conclusiones

1. Se desarrolló y verificó **Yuyanna Hub**, una aplicación multiplataforma en Flutter que centraliza las convocatorias de la incubadora Paqarina Wasi y automatiza la conformación de equipos multidisciplinarios, cumpliendo el objetivo general del proyecto sobre plataformas Android y web con una única base de código.
2. El **motor de emparejamiento** quedó implementado y probado: al publicarse un grupo, el sistema recorre el directorio real de usuarios, puntúa a cada candidato según cuántas de las habilidades faltantes cubre y notifica automáticamente a los diez mejores perfiles. La función de compatibilidad adoptada posee una propiedad de ordenamiento demostrable —la cobertura del déficit domina y el semestre solo desempata— y es *explicable*, pues cada invitación comunica qué habilidades concretas la motivaron. Con ello, el proyecto sustituye un proceso que hoy depende del círculo social del estudiante por un criterio de complementariedad verificable.
3. El **ciclo completo de formación de equipos** está operativo: borrador privado, publicación con emparejamiento, doble vía de ingreso con la asimetría correcta de decisión (el invitado decide sobre su invitación; el líder decide sobre las solicitudes) y cierre automático en cascada al completarse el cupo, con las invitaciones y solicitudes pendientes resueltas para no dejar notificaciones huérfanas.
4. La **restricción de costo cero** (RNF-02) fue el condicionante de diseño más influyente del proyecto. Determinó la ausencia de Cloud Functions —y con ello la ejecución del dominio en el cliente y el compromiso de escritura documentado en 6.5—, la sustitución de Firebase Storage por un Worker de Cloudflare sobre R2, y la estrategia de consultas de un solo campo para evitar índices compuestos. Cada una de estas decisiones se documentó con su justificación y su ruta de remediación, de modo que el compromiso asumido es explícito, acotado y reversible.
5. La **simplicidad arquitectónica deliberada** —sin paquete de gestión de estado, sin enrutador y sin generación de código— resultó adecuada para el alcance del semestre: el proyecto cierra con el analizador estático sin observaciones y las doce pruebas en verde, con la cobertura concentrada donde reside el riesgo real, que es la lógica de dominio.
6. Las desviaciones respecto del plan inicial no fueron omisiones sino **decisiones fundamentadas**. La más relevante desde la ingeniería fue reemplazar la similitud de Jaccard por una función aditiva, al constatar que el denominador de Jaccard penaliza a los perfiles más completos, justo lo contrario de lo que el problema requiere. La más relevante desde el impacto fue convertir el dominio institucional en una insignia de confianza en lugar de un muro de acceso, lo que incorpora al ecosistema a mentores y egresados sin diluir la identidad universitaria de la plataforma.
7. El sistema queda en condiciones de **piloto controlado** con Paqarina Wasi. Su evolución a producción abierta exige, en este orden, migrar la lógica de equipos a Cloud Functions con transacciones, completar la recomendación inversa y sustituir el secreto estático del Worker de subida por autorización basada en el token de Firebase.

13 Referencias

- Google LLC. *Flutter Documentation*. <https://docs.flutter.dev/>
- Google LLC. *Dart Language Tour*. <https://dart.dev/guides>
- Google LLC. *Cloud Firestore — Data model and best practices*. <https://firebase.google.com/docs/firestore>
- Google LLC. *Firebase Authentication*. <https://firebase.google.com/docs/auth>
- Google LLC. *Firebase Security Rules*. <https://firebase.google.com/docs/rules>
- Cloudflare, Inc. *Cloudflare Workers Documentation*. <https://developers.cloudflare.com/workers/>
- Cloudflare, Inc. *R2 Object Storage*. <https://developers.cloudflare.com/r2/>
- Google LLC. *Material Design 3*. <https://m3.material.io/>
- Sadalage, P. J. y Fowler, M. *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Addison-Wesley, 2012.
- Universidad Nacional de San Antonio Abad del Cusco. *Incubadora de Empresas Paqarina Wasi*.